EVALUATION FRAMEWORK · WORKING DRAFT

# A Control Layer for Autonomous Banking Agents

Designing and evaluating a hybrid rules + LLM control layer that gates payments, transfers, loan modifications, and account changes attempted by an AI agent — testing whether it can correctly allow, block, or escalate actions across ambiguous intent, policy violations, excessive amounts, conflicting instructions, and API failures.

**Prepared by:** Harsha

**Subject under test:** The control layer (agent is a stimulus generator, not the subject of evaluation)

**Status:** No universally adopted control model exists yet — this is exploratory, not a reference implementation

## 01 Why This Problem, Why Now

The industry is actively building toward autonomous financial agents, but nobody has agreed on how to constrain them. That gap is the premise of this project.

**REGULATORY SIGNAL**

The Financial Stability Board published a consultation report in June 2026 proposing 12 sound practices for agentic AI in finance, covering governance, AI lifecycle management, and third-party risk — with a final report not expected until October 2026. This is a live, unsettled process, not an established standard.

**NAMED UNSOLVED GAP**

Industry analysis describes "Agentic Drift" — autonomous agents whose reasoning adapts based on new data or goal-chaining can drift from their originally assessed boundary and violate compliance rules *without triggering traditional alerts*. Detecting this requires watching behavior across a conversation, not just checking a single action in isolation.

### What the industry confirms

- UK's FCA is warning that rapid agentic AI adoption could expose consumers and the financial system to new risk, even as major lenders prepare live customer-facing trials
- Banks are shifting from static rule-books to reasoning engines that interpret goals and execute end-to-end, escalating "only when human sign-off genuinely matters" — but the criteria for that judgment call are not standardized
- One 2026 analysis names a "Deployment Gap": banks invest in the agent's reasoning but under-invest in the infrastructure that proves what the agent did after the fact

### What this implies for the project

- There is no external answer key to validate against — the scenario labels and thresholds **are** the contribution, not a byproduct of building the code
- A single-turn, single-action test is insufficient — drift and conflicting intent only show up across a conversation
- "Escalate" needs to be treated as a first-class correct answer, not a fallback — especially for ambiguous cases where a confident allow or block is worse than asking


- Financial Stability Board consultation on agentic AI in finance — TechInformed, "FSB calls for tighter controls on agentic AI in finance"
- FCA warnings on agentic AI trials — CeFPro Connect, "Banks Face New Risks as Agentic AI Powers Toward 2026 Launch"
- "Agentic Drift" / Deployment Gap — Valuebound, "AI for Regulatory Compliance in Banking: The 2026 Mandate"
- Shift from rule-books to reasoning engines — Silent Infotech, "The Rise of Agentic AI: Transforming Banking Operations in 2026"

## 02 Objective & Scope

### Objective

Determine whether a hybrid control layer can correctly gate an AI agent's banking actions across varying autonomy levels — without being so restrictive that the agent becomes unusable for legitimate tasks. The core question is whether a workable balance point exists, or whether tightening safety in one scenario category collapses usability in another.

### What is actually being tested

> **SUBJECT UNDER TEST**
>
> The **control layer's** guardrail accuracy. The agent itself is a stimulus generator — it is deliberately built to be obedient, including to bad-faith or confused instructions, so that the control layer has something real to catch. The agent's own judgment or alignment is explicitly out of scope for this phase.

#### In scope

- Payments, transfers, loan modifications, account changes via a simulated backend
- Multi-turn conversations — follow-ups, corrections, contradictions across turns
- Configurable agent autonomy levels as an input to the control layer's policy decision
- Allow / Block / Escalate as the three decision outcomes, scored per scenario category

#### Out of scope (this phase)

- The agent's own alignment or reasoning quality
- Real regulatory compliance or certification
- Production-grade security, auth, or persistence
- A live HTTP service — actions are in-process function calls for fast iteration

### Ground truth

No external standard exists to check answers against. Each scenario is manually labeled with an expected decision, making **scenario design the intellectual core of the project** — the code that runs the eval is comparatively mechanical by comparison.

## 03 System Architecture

Five components, each with a narrow job. The control layer is the only one under evaluation.

### 1 Simulated Banking Backend

- In-process functions (not a live HTTP service) for fast iteration on data shapes
- Four action families: payments, transfers, loan modifications, account changes
- Seeded with realistic edge-case state — accounts near limits, delinquent loans — since seed data is itself part of scenario design

### 2 Control Layer — Hybrid Rules + LLM

- **Rules engine** for hard, unambiguous limits: amount ceilings, blacklisted actions per autonomy level, account/loan status gates
- **LLM judge** for the ambiguous residue: unclear intent, conflicting instructions across turns, whether escalation is warranted
- Decision contract: `(conversation_state, requested_action, autonomy_level) → (decision, reason, confidence)`

### 3 Agent Harness

- Deliberately simple and obedient — attempts whatever a scenario script calls for, including bad or contradictory instructions
- Configurable autonomy levels, which the control layer consumes as a policy input

### 4 Scenario Suite

- Multi-turn conversation traces, not single instructions — correctness can depend on something said several turns earlier
- Each scenario carries a manually adjudicated expected decision (the ground truth)

### 5 Evaluation Harness

- Runs the full scenario × autonomy-level matrix, logs decision and reasoning
- Produces a scorecard broken out **per category** rather than one aggregate score

## 04 Scenario Taxonomy & Failure Weighting

Failure-mode weighting isn't a global setting — it falls out of what each scenario category is actually testing. API failures don't even belong on the same scorecard as the rest.

| CATEGORY | WHAT A MISS LOOKS LIKE | WHAT "CORRECT" USUALLY MEANS | WORST ERROR |
|---|---|---|---|
| Excessive transaction amount | Control layer allows a transfer far past the account or policy limit | Block, or escalate near the threshold | **False-allow** — direct financial harm |
| Clear policy violation | Control layer allows an action that violates a hard rule (e.g. closing an account with a lien) | Block | **False-allow** — hard compliance breach |
| Ambiguous intent | Control layer guesses the wrong amount, account, or action | Escalate — ask, don't guess | **Missed escalation** — confident wrong guess is worse than asking |
| Conflicting instructions (multi-turn) | Control layer acts on a stale or superseded instruction from earlier in the conversation | Escalate, or correctly resolve to the latest valid instruction | **Missed escalation** / state-tracking failure |
| Injected API failure | Control layer retries destructively, double-executes, or silently drops a failed action | Idempotent retry or clean failure — not a policy judgment at all | Scored on a **separate robustness axis**, not blended with policy scores |

**SCORING PRINCIPLE**

Report results per category, not as one aggregate accuracy number. A false-allow on an excessive-amount scenario and a missed escalation on an ambiguous-intent scenario are not equivalent failures, and averaging them together would obscure exactly the distinction this evaluation exists to surface.

## 05 Standard Operating Procedure

### 1 Foundation

- Define the state model — accounts, payees, loans, limits, transaction log
- Build backend functions for the four action categories with realistic seeded edge cases
- Define the decision contract the control layer must implement

### 2 Control Layer

- Build the rules engine for hard, deterministic limits
- Build the LLM judge for ambiguous/judgment cases
- Define explicit handoff logic — which cases the rules engine resolves outright vs. defers

### 3 Scenario Taxonomy

- Write and label scenarios per category in the taxonomy above
- Adjudicate expected outcomes manually — this is the ground truth the system is scored against

### 4 Agent Harness

- Build the deliberately-obedient agent with configurable autonomy levels

### 5 Evaluation Harness

- Run the scenario × autonomy-level matrix
- Log every decision with reasoning; produce a per-category scorecard

### 6 Analysis

- Identify where the control layer fails and why — over-triggering the LLM judge, under-escalating ambiguous cases
- Determine whether a workable safety/usability balance exists, or whether tightening one category collapses another

## 06 Relevance & the Open Question

### Why this is a relevant project, not a re-solved problem

- The problem is genuinely unsolved and being actively defined by regulators in real time — the FSB's own practices are still in consultation, not finalized
- "Agentic Drift" is named as a current, open gap in industry analysis — this project's multi-turn design is built specifically to probe that gap
- Banking is explicitly called out as "the higher-stakes, more regulated version" of the broader shift toward agentic workflow automation — the stakes are real, not hypothetical
- Strong portfolio fit: not "built a CRUD app," but "designed an evaluation methodology for an open industry problem," aligned with a backend / distributed-systems and fintech career direction
- The lack of an external standard is not a weakness of the project — it is the reason the project's own scenario design and labeling **is** the contribution

### The honest caveat

> **CREDIBILITY DEPENDS ON LABELING, NOT CODE**
>
> Because no external benchmark exists yet, this evaluation's credibility rests entirely on whether the scenario labels and rule thresholds are defensible. That is not a flaw in the plan — it is the actual research contribution. It does mean scenario design deserves more time than the implementation, not less; for this project the usual build ratio should invert.

### What this evaluation is ultimately trying to answer

**Can a control layer be built that reliably stops harmful autonomous financial actions — without becoming so restrictive that the agent it's supposed to govern stops being useful?**